

PRACTICALWEEK-3

Session-3

Task1: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
Priyavallika@LAPTOP-D2M8NVKHBH:~/OSSP/practical/week3/task1$ cd ~/OSSP
Priyavallika@LAPTOP-D2M8NVKHBH:~/OSSP$ mkdir -p practical/week3/task1
Priyavallika@LAPTOP-D2M8NVKHBH:~/OSSP$ cd practical/week3/task1
Priyavallika@LAPTOP-D2M8NVKHBH:~/OSSP/practical/week3/task1$ pwd
/home/Priyavallika/OSSP/practical/week3/task1
Priyavallika@LAPTOP-D2M8NVKHBH:~/OSSP/practical/week3/task1$ nano fork_process.c
Priyavallika@LAPTOP-D2M8NVKHBH:~/OSSP/practical/week3/task1$ gcc fork_process.c -o fork_process
Priyavallika@LAPTOP-D2M8NVKHBH:~/OSSP/practical/week3/task1$ ./fork_process
Parent process started.
Parent PID: 1146

--- Parent Process ---
Parent PID: 1146
Child PID: 1147
Parent is waiting for child...

--- Child Process ---
Child PID: 1147
Parent PID (PPID): 1146
Child process is running...
Child process terminated.
Parent process terminated.
Priyavallika@LAPTOP-D2M8NVKHBH:~/OSSP/practical/week3/task1$
```

Observation: The program was executed successfully using the fork() system call. A parent process with PID 1146 created a child process with PID 1147. The child's PPID was 1146, confirming the parent-child relationship. The parent waited while the child executed, and after the child completed its execution, the parent process also terminated.

Result: The experiment successfully demonstrated process creation, PID and PPID identification, parent-child execution, and process termination using fork() and wait().

Task2: Design an experiment Observe process state transitions (Ready, Running, Waiting, Terminated) using ps, top, and /proc, and document the observations.

```
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3$ cd ~/OSSP/practical/week3
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3$ mkdir -p task2
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3$ cd task2
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3/task2$ pwd
/home/priyavallika/OSSP/practical/week3/task2
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3/task2$ nano process_states.c
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3/task2$ gcc process_states.c -o process_states
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3/task2$ ./process_states &
[1] 1254
Process started.
PID: 1254
PPID: 488
Process is running.
Process will now enter waiting state.
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3/task2$ ps -p 1254 -o pid,ppid,stat,cmd
  PID   PPID  STAT  CMD
  1254   488    S    ./process_states
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3/task2$ grep "^State:" /proc/1254/status
State:    S (sleeping)
Priyavallika@LAPTOP-D2M8NKMBH:~/OSSP/practical/week3/task2$ top
top - 08:21:48 up 37 min,  1 user,  load average: 0.00, 0.01, 0.00
Tasks: 28 total,  1 running, 27 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 99.9 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  5778.6 total,   5167.9 free,   380.7 used,   230.0 buff/cache
MiB Swap:  2048.0 total,   2048.0 free,    0.0 used,   5236.3 avail Mem
```

PID	USER	PR	NI	VIRT	RES
87	root	20	0	35348	12304
1	root	20	0	24192	15388
2	root	20	0	3180	2204
6	root	20	0	3180	2044
49	root	19	-1	50400	16664
81	systemd+	20	0	22416	14488
107	root	20	0	2888	1944
108	root	20	0	4472	3108
109	message+	20	0	8820	5416
115	root	20	0	44096	29880
129	root	20	0	18440	9452
181	syslog	20	0	220548	5904
190	root	20	0	5492	2704
257	_chrony	20	0	23684	11064
265	root	20	0	123460	32548
279	_chrony	20	0	12096	2464
404	root	20	0	3184	1108
407	root	20	0	3200	1248
408	madhuli+	20	0	6268	5556
410	root	20	0	8644	5544
453	madhuli+	20	0	22336	13516
456	madhuli+	20	0	22912	4088
485	madhuli+	20	0	6136	5448

Observation: The process was successfully monitored using ps, top, and /proc. The process was observed in the running and waiting states during execution. The S state indicated that the process was sleeping/waiting. After using the kill command, the process was terminated successfully.

Result: The different process states were successfully observed and verified using Linux process-monitoring commands.